

IA PARA DEVS

PROCESSAMENTO DE LINGUAGEM NATURAL

AULA 03

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	17
O QUE VOCÊ VIU NESTA AULA?	22
REFERÊNCIAS.....	23

EMANIP

O QUE VEM POR AÍ?

Bem-vindos e bem-vindas à aula sobre tokenização com NLTK e remoção de stop words! Vamos aprender o conceito de tokenização e utilizar uma das melhores bibliotecas do mercado, a NLTK, e adentrar em seus componentes e observar como pode ser produtivo para nosso projeto a tokenização de sentenças. Após isso, vamos observar problemas que poderão ser resolvidos com a remoção das stop words do nosso dataset e vamos implementar diversas maneiras de contornar isso. Por fim, vamos reanalisar os gráficos que fizemos na última aula e implementar melhorias neles.

HANDS ON

Nesta aula continuamos analisando o nosso classificador de sentimentos, porém implementando técnicas de tokenização com a biblioteca NLTK e remoção de stop words.

Para pegarmos as palavras mais frequentes, vamos precisar calcular a frequência das palavras, e para isso, vamos usar uma função dentro do nltk chamada FreqDist

Vamos fazer um exemplo aqui para entender como funciona:

```
import nltk

corpus = ["Muito bom este produto", "Muito ruim este produto"]

frequencia = nltk.FreqDist(corpus)

frequencia
```

```
⇒ FreqDist({'Muito bom este produto': 1, 'Muito ruim este produto': 1})
```

Supondo que passemos para o método nosso corpus textual, frase a frase ele vai calcular a frequência de cada frase. Para funcionar corretamente, nós vamos precisar mandar cada palavra como se fosse um item no array.

Para isso, podemos utilizar a lib tokenize dentro do nltk.

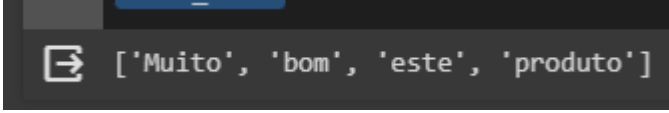
Vamos pegar nossa primeira frase do corpus textual e vamos tokenizá-la:

```
from nltk import tokenize

frase = "Muito bom este produto"

token_por_espaco = tokenize.WhitespaceTokenizer()
```

```
token_frase = token_por_espaco.tokenize(frase)
token_frase
```



```
['Muito', 'bom', 'este', 'produto']
```

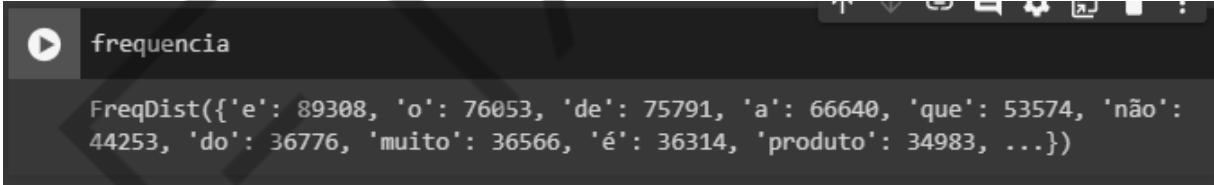
Ele vai tokenizar a frase.

Mas pensando no nosso dataset, como nós podemos tokenizar a base toda?

Se lembra que nós já fizemos a junção de todas as palavras do dataset em uma string gigante, basicamente é só fazer a mesma coisa pra essa variável que já temos aqui.

```
token_por_espaco = tokenize.WhitespaceTokenizer()
token_dataset = token_por_espaco.tokenize(todas_palavras)
frequencia = nltk.FreqDist(token_dataset)
```

E com isso nós já calculamos a frequência também das palavras:



```
frequencia
FreqDist({'e': 89308, 'o': 76053, 'de': 75791, 'a': 66640, 'que': 53574, 'não': 44253, 'do': 36776, 'muito': 36566, 'é': 36314, 'produto': 34983, ...})
```

Visualizando a variável, já conseguimos ter uma ideia dos dados, mas dá para melhorar. Para isso, vamos jogar em um DataFrame do Pandas pra conseguirmos trabalhar melhor.

```
dataframe_frequencia = pd.DataFrame({"Palavra": list(frequencia.keys()),
                                     "Frequencia": list(frequencia.values())})
```

```
dataframe_frequencia.head()
```



	Palavra	Frequencia
0	Bem	413
1	macio	216
2	e	89308
3	felpudo...recomendo.	1
4	Preço	334

Figura 1 – Resultado 1
Fonte: Elaborado pelo autor (2024)

E com isso nós já conseguimos usar métodos do Pandas como o `nlargest` que pega os N primeiros mais frequentes dado uma coluna, assim:

```
dataframe_frequencia.nlargest(columns = "Frequencia", n = 10)
```



	Palavra	Frequencia
2	e	89308
20	o	76053
17	de	75791
39	a	66640
50	que	53574
33	não	44253
23	do	36776
43	muito	36566
60	é	36314
21	produto	34983

Figura 2 – Resultado 2
Fonte: Elaborado pelo autor (2024)

Aqui já vemos algumas stop words, mas vamos visualizar esses dados graficamente com a biblioteca Seaborn que já vem instalada no colab.

```
import seaborn as sns

plt.figure(figsize=(12,8))

ax = sns.barplot(data = dataframe_frequencia.nlargest(columns = "Frequencia", n =
10),

                x = "Palavra", y = "Frequencia", color = "lightblue")

ax.set(ylabel = "Contagem")

plt.show()
```

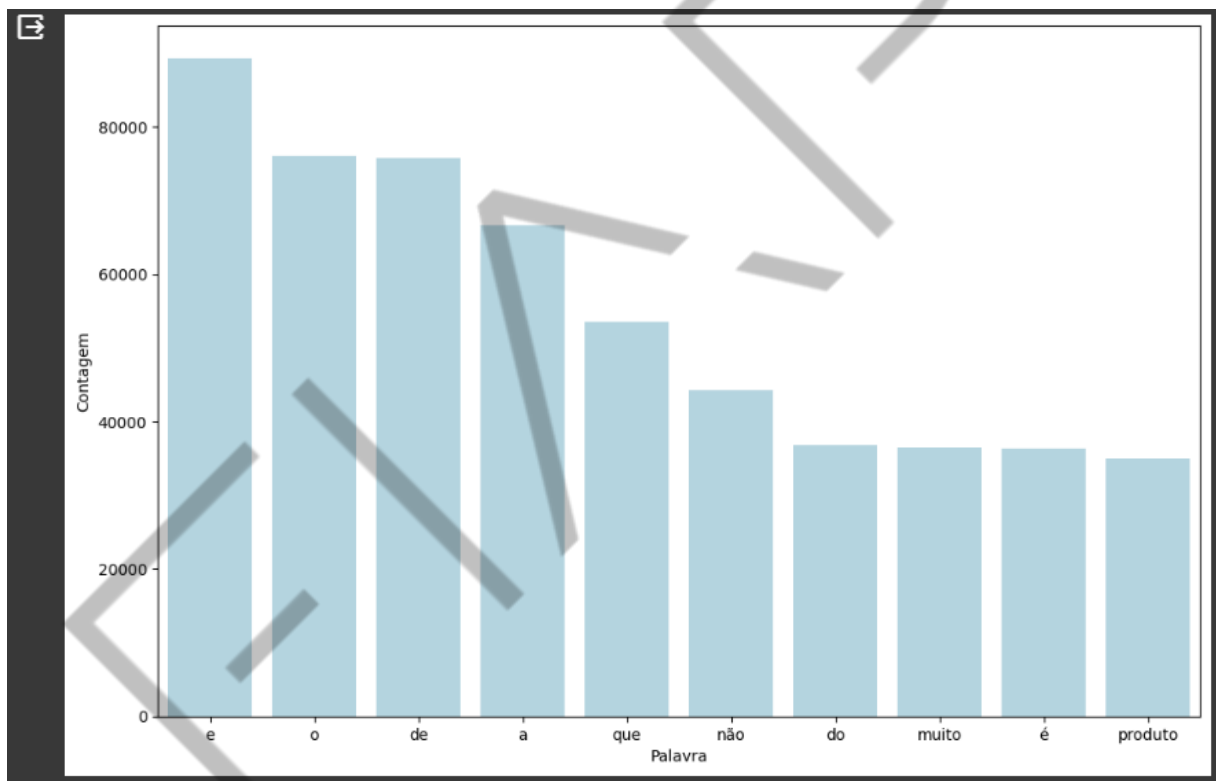


Figura 3 - Biblioteca Seaborn
Fonte: Elaborado pelo autor (2024)

Com esse gráfico já dá pra ver a diferença da quantidade de palavras.

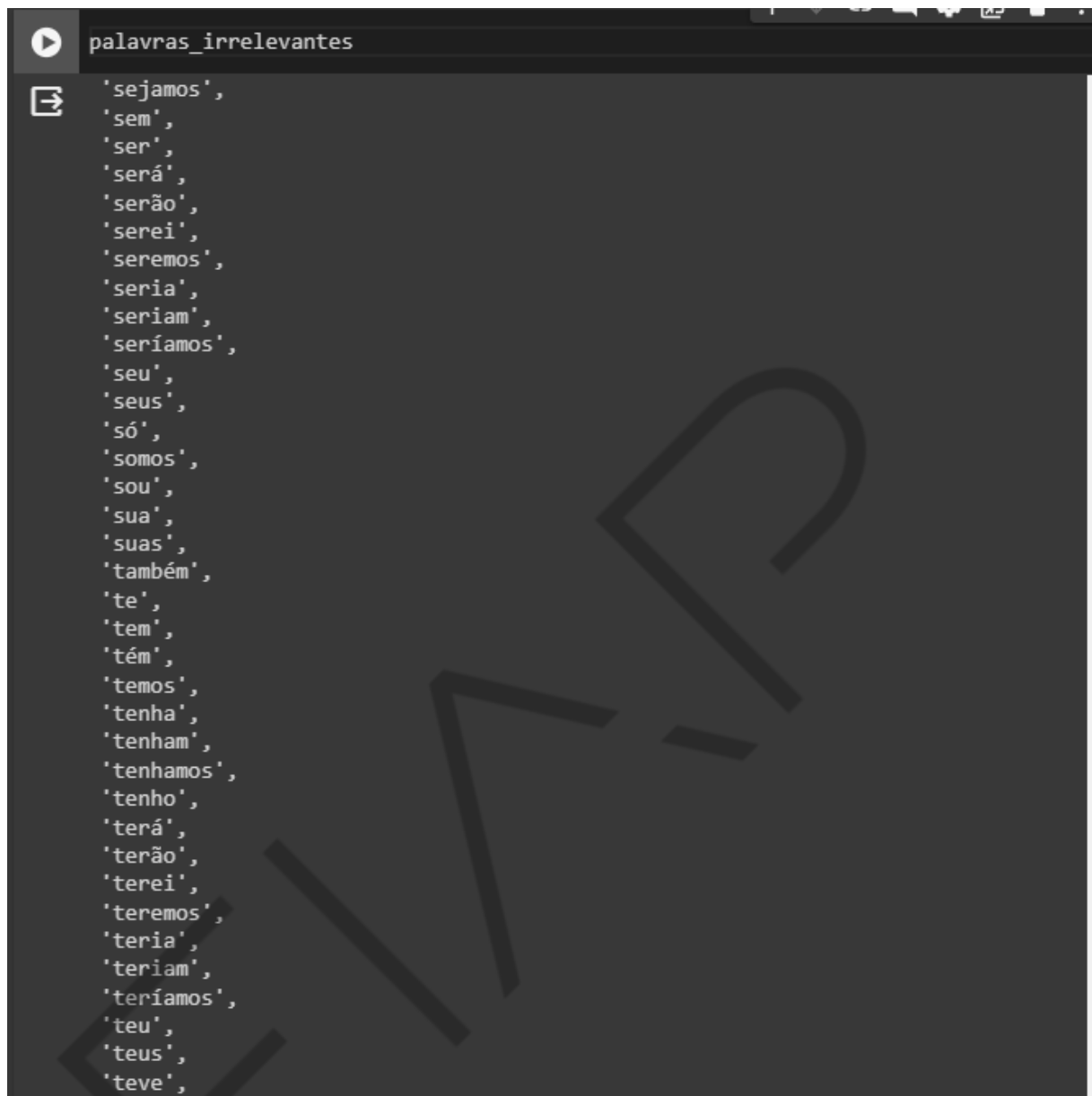
Vamos criar um método para criar esse gráfico com alguns parâmetros:

```
def grafico(dados, coluna_texto, quantidade):  
    todas_palavras = ' '.join([texto for texto in dados[coluna_texto]])  
    token_frase = token_por_espaco.tokenize(todas_palavras)  
    frequencia = nltk.FreqDist(token_frase)  
    dataframe_frequencia = pd.DataFrame({"Palavra": list(frequencia.keys()),  
                                         "Frequencia": list(frequencia.values())})  
    dataframe_frequencia = dataframe_frequencia.nlargest(columns = "Frequencia",  
                                                         n = quantidade)  
  
    plt.figure(figsize=(12,8))  
    ax = sns.barplot(data = dataframe_frequencia,  
                    x = "Palavra", y = "Frequencia", color = "lightblue")  
    ax.set(ylabel = "Contagem")  
    plt.show()
```

Tudo bem, agora que vimos como identificar e analisar melhor os nossos dados, percebemos a presença de stop words. Sendo assim, como podemos retirá-las para não afetar nossa tomada de decisão?

Alguém já mapeou isso pra gente, e essa lista tá dentro do nltk, e conseguimos removê-las com o nltk.

```
palavras_irrelevantes = nltk.corpus.stopwords.words("portuguese")
```



```
palavras_irrelevantes
'sejamos',
'sem',
'ser',
'será',
'serão',
'serei',
'seremos',
'seria',
'seriam',
'seríamos',
'seu',
'seus',
'só',
'somos',
'sou',
'sua',
'suas',
'também',
'te',
'tem',
'tém',
'temos',
'tenha',
'tenham',
'tenhamos',
'tenho',
'terá',
'terão',
'terei',
'teremos',
'teria',
'teriam',
'teríamos',
'teu',
'teus',
'teve',
```

Figura 4 – Lista do nltk
Fonte: Elaborado pelo autor (2024)

Se visualizarmos isso nós já encontramos algumas palavras que temos na nossa base. Agora precisamos passar por cada frase do nosso dataset e retirar as stop words dela:

```

frase_processada = list()

for avaliacao in avaliacoes.review_text:

    nova_frase = list()

    palavras_texto = token_por_espaco.tokenize(avaliacao)

    for palavra in palavras_texto:

        if palavra not in palavras_irrelevantes:

            nova_frase.append(palavra)

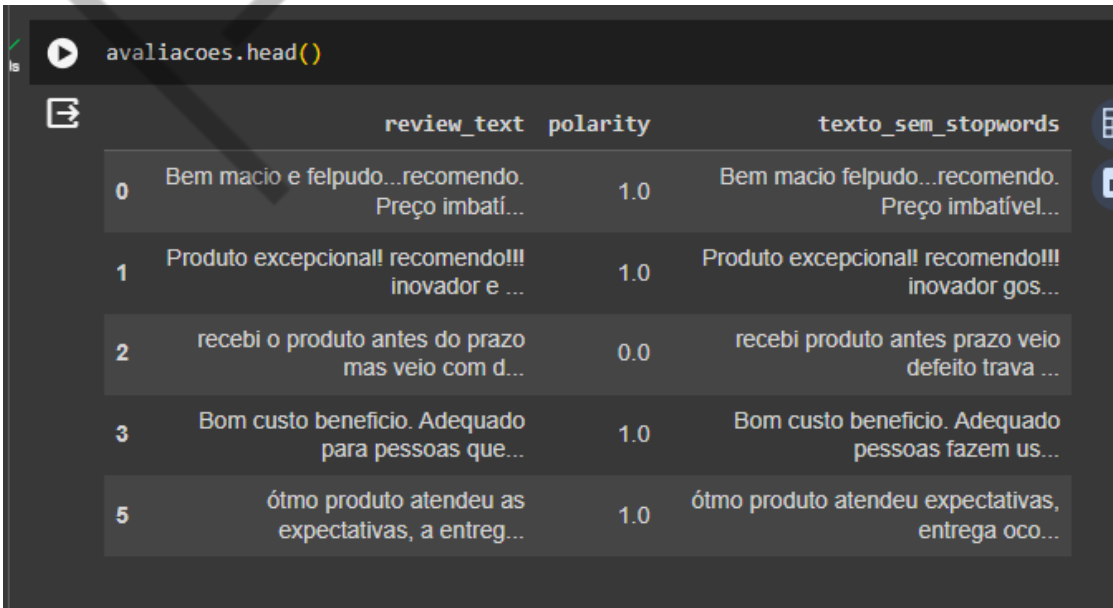
    frase_processada.append(' '.join(nova_frase))

avaliacoes["texto_sem_stopwords"] = frase_processada

```

Neste código basicamente estamos iterando a base, para cada frase estamos tokenizando e passando palavra a palavra se ela existe na lista de palavras irrelevantes. Se sim, estamos descartando e montando a frase novamente, mas como ela está em lista nós demos um join nela novamente e colocamos na lista frase_processada, no fim nós só adicionamos a nova coluna para o dataframe.

```
avaliacoes.head()
```



	review_text	polarity	texto_sem_stopwords
0	Bem macio e felpudo...recomendo. Preço imbatível...	1.0	Bem macio felpudo...recomendo. Preço imbatível...
1	Produto excepcional! recomendo!!! inovador e ...	1.0	Produto excepcional! recomendo!!! inovador gos...
2	recebi o produto antes do prazo mas veio com d...	0.0	recebi produto antes prazo veio defeito trava ...
3	Bom custo beneficio. Adequado para pessoas que...	1.0	Bom custo beneficio. Adequado pessoas fazem us...
5	ótimo produto atendeu as expectativas, a entreg...	1.0	ótimo produto atendeu expectativas, entrega oco...

Figura 5 – Avaliações

Fonte: Elaborado pelo autor (2024)

```
grafico(avaliacoes, "texto_sem_stopwords", 20)
```

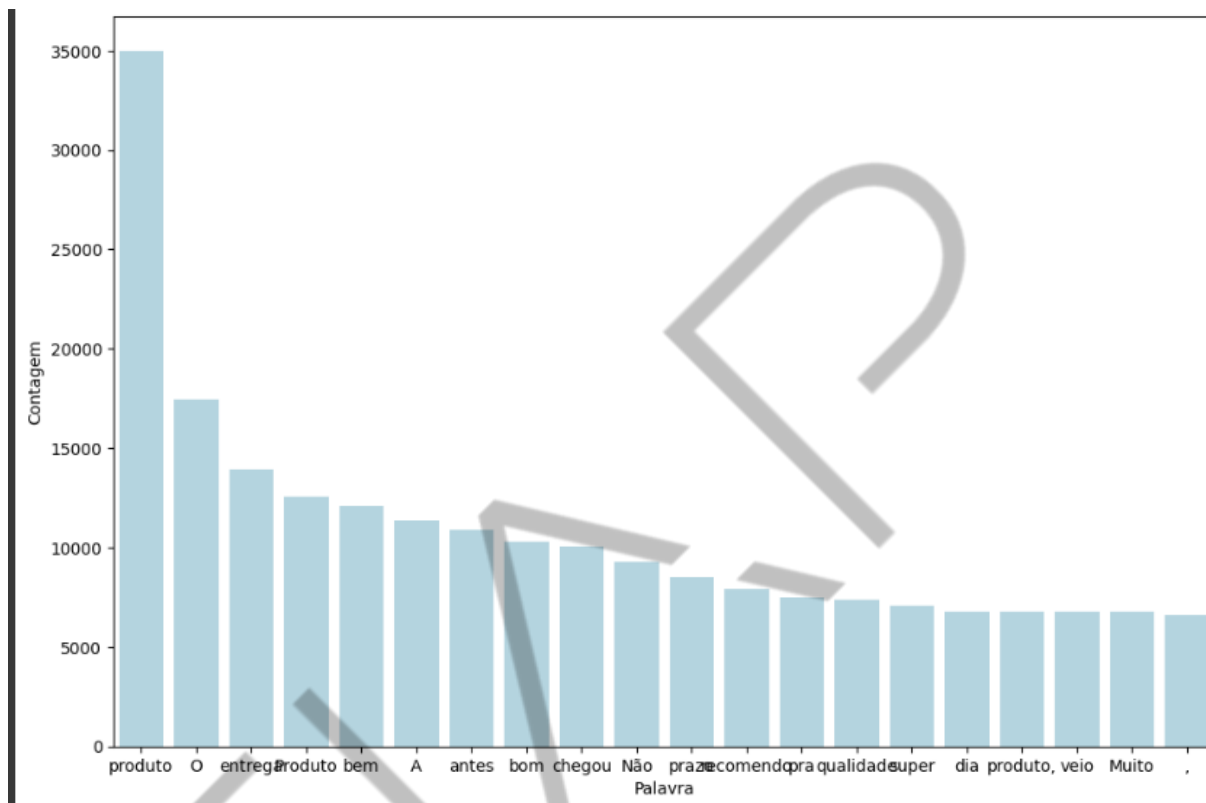


Figura 6 – Gráfico de avaliações
Fonte: Elaborado pelo autor (2024)

O produto já é nossa palavra mais relevante, mas conseguimos observar algumas stop words que não estão na lista do nltk, como o “O”, o “A” e assim vai.

Podemos fazer uma tratativa legal também, que é a retirada da pontuação, como por exemplo a frase:

O produto é bom.

Muito bom, esse produto.

Quando formos analisar a palavra bom, por exemplo, elas virão como duas palavras:

bom.

bom,

O que não é interessante para nossa análise. Para isso, vamos usar uma função do nltk dentro do tokenize, mas antes precisamos saber qual vai ser a pontuação que vamos querer retirar do nosso dataset.

Podemos pegar isso da biblioteca string:

```
from string import punctuation
punctuation
```

```
'!"#$%&\ '()*+,-./:;<=>?@[\\]^_`{|}~'
```

```

pontuacao = list()

for ponto in punctuation:
    pontuacao.append(ponto)

pontuacao

```

Com isso, temos uma lista da nossa pontuação separada.

Agora, se lembram que nós removemos as stop words da nossa base? nós vamos fazer isso de novo, porém vamos adicionar a nossa lista de stop words a nossa pontuação.

Vamos fazer a mesma coisa, porém vamos fazer a partir da nossa coluna do texto sem stopwords:

```
pontuacao_stopwords = pontuacao + palavras_irrelevantes

frase_processada = list()

for avaliacao in avaliacoes.texto_sem_stopwords:

    nova_frase = list()

    palavras_texto = token_pontuacao.tokenize(avaliacao)
```

```
for palavra in palavras_texto:

    if palavra not in pontuacao_stopwords:

        nova_frase.append(palavra)

frase_processada.append(' '.join(nova_frase))

avaliacoes["texto_sem_stopwords_e_pontuacao"] = frase_processada
```

Agora, dando uma olhada em um item aqui, podemos observar que já retirou as pontuações.

```
grafico(avaliacoes, "texto_sem_stopwords_e_pontuacao", 20)
```

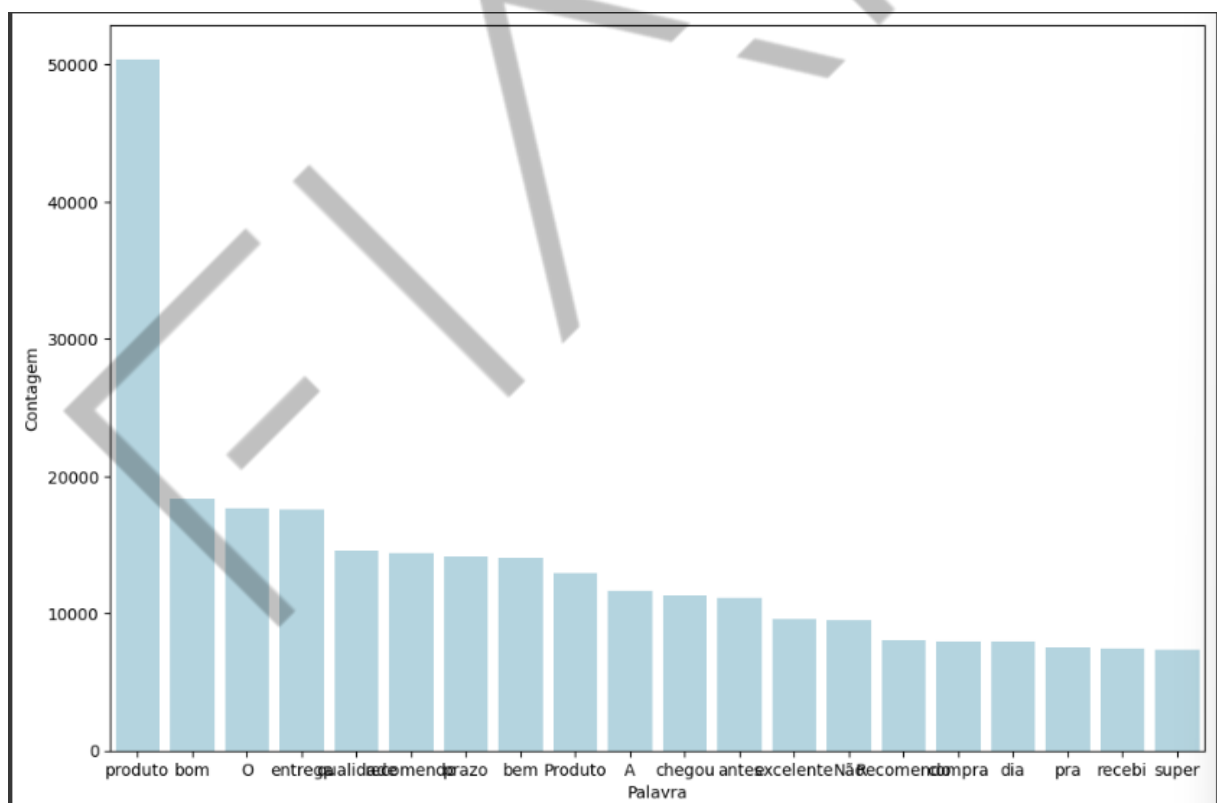


Figura 7 – Gráfico de avaliações atualizado
Fonte: Elaborado pelo autor (2024)

Para tirar o acento de palavras com é, ã, etc, podemos usar a biblioteca `unidecode`.

```
!pip install unicode
```

```
sem_acentos = [unicode.unidecode(texto) for texto in  
avaliacoes.texto_sem_stopwords_e_pontuacao]
```

```
stopwords_sem_acento = [unicode.unidecode(texto) for texto in  
pontuacao_stopwords]
```

Agora vamos retirar as stopwords sem acento da nossa base com o mesmo código de antes mudando algumas coisas.

```
avaliacoes["texto_sem_stopwords_e_pontuacao_e_acentos"] = sem_acentos  
  
frase_processada = list()  
for avaliacao in avaliacoes.texto_sem_stopwords_e_pontuacao_e_acentos:  
    nova_frase = list()  
    palavras_texto = token_pontuacao.tokenize(avaliacao)  
    for palavra in palavras_texto:  
        if palavra not in stopwords_sem_acento:  
            nova_frase.append(palavra)  
    frase_processada.append(' '.join(nova_frase))  
  
avaliacoes["texto_sem_stopwords_e_pontuacao_e_acentos"] = frase_processada
```

Aqui já visualizamos que já retirou os acentos.

Agora vamos treinar o modelo novamente para ver se melhorou a acurácia:

```
print(treinar_modelo(avaliacoes, "texto_sem_stopwords_e_pontuacao_e_acentos",  
"polarity"))
```

Vamos plotar nossa nuvem de palavras também.

```
word_cloud_neg(avaliacoes, "texto_sem_stopwords_e_pontuacao_e_acentos")
```

```
word_cloud_pos(avaliacoes, "texto_sem_stopwords_e_pontuacao_e_acentos")
```

Mais uma coisa que podemos arrumar, é as letras maiúsculas usando o método lower.

```
frase_processada = list()
for avaliacao in avaliacoes.texto_sem_stopwords_e_pontuacao_e_acentos:
    nova_frase = list()
    avaliacao = avaliacao.lower()
    palavras_texto = token_pontuacao.tokenize(avaliacao)
    for palavra in palavras_texto:
        if palavra not in stopwords_sem_acento:
            nova_frase.append(palavra)
    frase_processada.append(' '.join(nova_frase))

avaliacoes["texto_sem_stopwords_e_pontuacao_e_acentos_minusculo"] = frase_processada
```

Já conseguimos ver que a última coluna está toda minúscula, agora vamos ver nosso gráfico pra ver se ele sumiu as stop words maiúsculas.

```
word_cloud_neg(avaliacoes,  
"texto_sem_stopwords_e_pontuacao_e_acentos_minusculo")
```

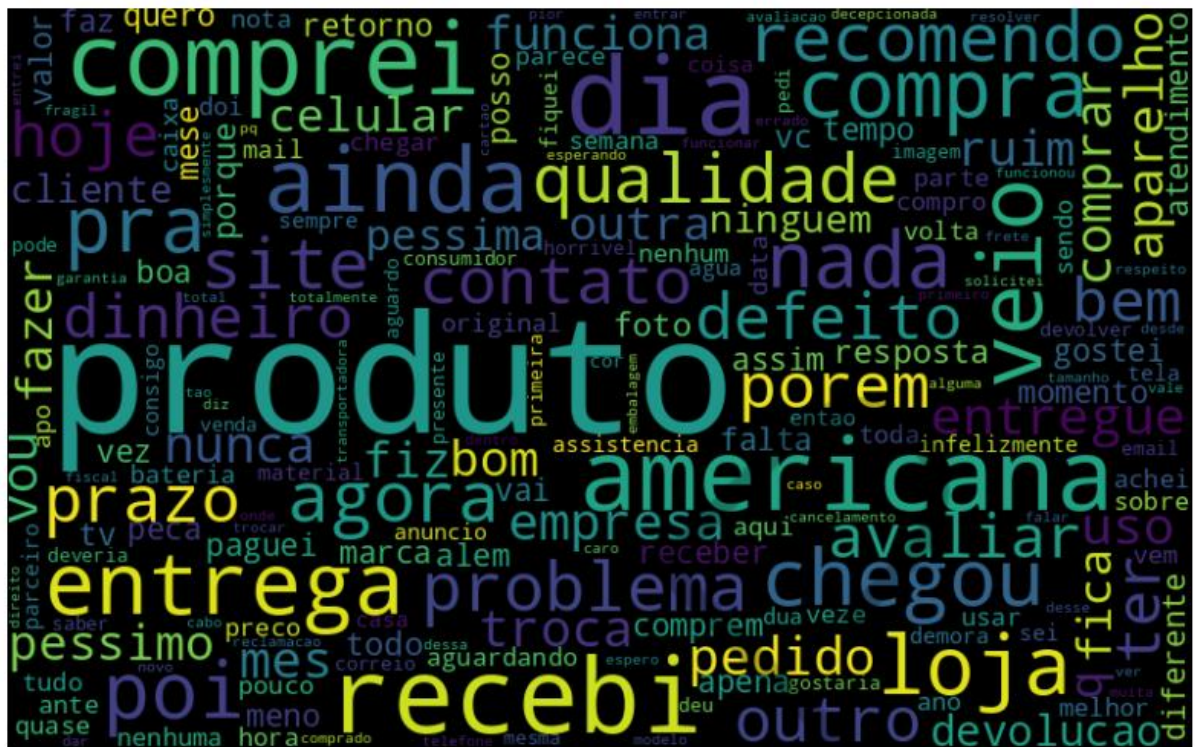


Figura 8 – Nuvem de palavras
Fonte: Elaborado pelo autor (2024)

```
word_cloud_pos(avaliacoes,  
"texto_sem_stopwords_e_pontuacao_e_acentos_minusculo")
```

SAIBA MAIS

A tokenização é um componente essencial do processamento de linguagem natural (PLN) que desempenha um papel crucial na manipulação e análise de texto. Consiste em dividir um texto em unidades menores chamadas tokens, que podem variar desde caracteres individuais até palavras ou frases completas. Essa técnica é amplamente utilizada em várias aplicações de PLN para preparar o texto para análises mais aprofundadas.

A tokenização, no âmbito do Processamento de Linguagem Natural (PLN), desempenha um papel crucial ao transformar textos complexos em unidades mais gerenciáveis, conhecidas como tokens. Essa técnica multifacetada é projetada com vários objetivos principais, cada um contribuindo para uma compreensão mais profunda e eficiente do conteúdo textual.

Facilitação da Análise: a principal meta da tokenização é simplificar a análise de texto. Ao dividir um documento extenso em unidades menores, a complexidade é reduzida, permitindo uma investigação mais detalhada. Isso é particularmente vital em tarefas como análise de sentimento, onde a compreensão de palavras individuais é essencial para extrair nuances emocionais.

Padronização do Texto: outro objetivo fundamental é a padronização do texto. A tokenização transforma o conteúdo em uma forma mais uniforme, facilitando a aplicação de técnicas de PLN. Essa uniformidade é crucial para garantir consistência nas análises e facilitar a comparação entre diferentes textos.

Redução de Dimensionalidade: ao converter um texto em tokens, a tokenização contribui para a redução da dimensionalidade do problema. Essa simplificação torna o processamento computacional mais eficiente, permitindo que os algoritmos de PLN operem de maneira mais rápida e eficaz em grandes volumes de texto.

Preparação para Tarefas Específicas: cada tarefa de PLN possui requisitos específicos, e a tokenização é projetada para preparar o texto de maneira adequada. Seja para classificação de texto, extração de informações ou geração de resumos, a identificação e segmentação de unidades linguísticas individuais é a base para análises mais avançadas.

Considerações Adicionais: para atingir esses objetivos, a tokenização se desdobra em métodos específicos. A tokenização de palavras, que divide o texto em unidades individuais, é a mais comum, enquanto a tokenização de frases considera o contexto mais amplo. A de caracteres, embora menos frequente, é aplicável em cenários específicos.

No entanto, a tokenização enfrenta desafios, como a ambiguidade contextual, onde o significado de palavras pode variar conforme o contexto, e a necessidade de adaptar abordagens à diversidade linguística.

Em resumo, a tokenização não é apenas uma técnica de pré-processamento, mas uma habilidade fundamental no arsenal de ferramentas de PLN, capacitando análises mais avançadas e proporcionando insights significativos a partir de grandes volumes de texto.

O Natural Language Toolkit, amplamente conhecido pela sigla NLTK, é uma biblioteca poderosa e abrangente em Python projetada para facilitar a manipulação e análise de dados linguísticos. Criado pela comunidade de pesquisa em Processamento de Linguagem Natural (PLN), o NLTK tornou-se um recurso essencial para pessoas desenvolvedoras, pesquisadores e entusiastas que buscam explorar e entender a linguagem humana por meio de dados textuais.

Origens e Desenvolvimento

O NLTK teve seu início como um projeto acadêmico na Universidade de Tecnologia de Chalmers, na Suécia, liderado por Steven Bird e Edward Loper. A primeira versão foi lançada em 2001, e desde então, o NLTK cresceu exponencialmente em popularidade, tornando-se uma ferramenta de referência em ambientes acadêmicos e industriais.

Funcionalidades Principais

- **Tokenização e Segmentação:** Uma das funcionalidades centrais do NLTK é a tokenização, que envolve a divisão de textos em palavras ou frases individuais. Além disso, o NLTK oferece ferramentas para segmentação de textos em unidades significativas, permitindo uma análise mais granular.
- **Processamento de Texto:** O NLTK fornece uma variedade de ferramentas para processamento de texto, incluindo stemming (redução de palavras à

sua forma base), lematização (normalização de palavras para suas formas de dicionário) e remoção de stop words (palavras comuns que não contribuem significativamente para o significado).

- **Classificação e Análise de Sentimentos:** Para tarefas de classificação de texto e análise de sentimentos, o NLTK oferece módulos específicos que simplificam a construção de modelos preditivos, permitindo a categorização eficiente de textos em classes predefinidas e a análise da polaridade emocional.
- **Recursos Linguísticos:** O toolkit é acompanhado por uma ampla variedade de recursos linguísticos, incluindo corpora (conjuntos de textos), lexicons (dicionários) e gramáticas, que podem ser explorados para uma compreensão mais aprofundada de estruturas linguísticas e vocabulário.
- **Processamento de Árvores Sintáticas:** O NLTK é capaz de lidar com análise sintática, incluindo a manipulação de árvores sintáticas, o que é essencial para tarefas como parsing e compreensão da estrutura gramatical de frases.
- **Comunidade e Suporte:** O NLTK, sendo um projeto de código aberto, beneficia-se de uma comunidade ativa de desenvolvedores e pesquisadores. Além da documentação abrangente, há uma variedade de tutoriais, livros e cursos online que ajudam os usuários a aproveitar ao máximo as capacidades do NLTK.
- **Integração e Continuidade:** O NLTK é frequentemente utilizado em conjunto com outras bibliotecas populares, como NumPy e Pandas, para análise de dados e manipulação eficientes. Sua constante atualização e manutenção garantem que ele permaneça relevante e adaptável às demandas evolutivas do campo de PLN.

O Natural Language Toolkit não é apenas uma biblioteca, mas uma ferramenta essencial que abrange uma gama diversificada de funcionalidades para Processamento de Linguagem Natural. Seu papel central na comunidade de PLN e sua flexibilidade fazem do NLTK uma escolha valiosa para aqueles que buscam explorar, entender e aplicar técnicas avançadas de análise de texto.

A remoção de stop words é uma etapa fundamental e meticulosa no pré-processamento de texto no campo do PLN. Esse procedimento visa filtrar palavras comuns, mas frequentemente irrelevantes, que não contribuem para o significado essencial de um texto. O conceito de "stop words" refere-se a termos como artigos, preposições e conjunções, cuja presença em um corpus pode adicionar ruído semântico e aumentar a complexidade computacional sem agregar valor à análise.

Motivações e Objetivos

- **Aprimoramento da Eficiência Computacional:** A remoção de stop words visa otimizar o processamento computacional, reduzindo a carga de trabalho associada ao manuseio de palavras que, por si só, têm pouco valor semântico.
- **Foco na Informação Relevante:** A eliminação de stop words concentra a atenção nas palavras mais informativas, permitindo uma análise mais precisa e efetiva do conteúdo essencial do texto.
- **Prevenção da Distorção Semântica:** Ao eliminar palavras comuns que ocorrem frequentemente em diversos contextos, a remoção de stop words ajuda a evitar a distorção semântica, proporcionando uma representação mais precisa do conteúdo textual.

Métodos e Desafios na Remoção de Stop Words

- **Construção de Listas Personalizadas:** A criação de listas personalizadas de stop words é uma abordagem comum, permitindo que os usuários adaptem a remoção de acordo com o contexto específico da aplicação. Isso pode envolver a consideração de termos específicos de domínio ou peculiaridades linguísticas.
- **Integração com Bibliotecas de PLN:** Bibliotecas como o NLTK e o spaCy oferecem métodos convenientes para a remoção de stop words, facilitando a implementação desse processo em diferentes projetos de PLN.
- **Desafios na Ambiguidade:** A remoção de stop words pode apresentar desafios em casos de ambiguidade, onde certas palavras podem ser tanto stop words quanto termos informativos, dependendo do contexto.

- **Idiomas e Variações Linguísticas:** A aplicação de remoção de stop words pode variar entre idiomas devido a diferenças gramaticais e culturais, exigindo ajustes para garantir eficácia.

Impacto na Análise de Texto

- **Preservação do Contexto:** A remoção cuidadosa de stop words visa preservar o contexto semântico, eliminando apenas os elementos redundantes e mantendo a integridade do significado.
- **Filtragem Adaptativa:** A capacidade de adaptar a lista de stop words às peculiaridades do corpus permite uma filtragem mais eficaz, evitando a perda de informações valiosas.

Limitações

- **Sensibilidade ao Contexto Cultural:** Em projetos com contextos culturais específicos, a remoção de stop words deve ser realizada com sensibilidade para evitar distorções ou perda de nuances culturais.
- **Preservação da Diversidade Linguística:** A remoção de stop words pode afetar a diversidade linguística, e, portanto, deve ser aplicada com discernimento, especialmente em ambientes multilíngues.

A remoção de stop words é uma etapa crítica no processamento de linguagem natural, contribuindo para análises mais eficientes e precisas. Ao compreender as motivações por trás dessa prática, os métodos de implementação e os desafios associados, os praticantes de PLN podem aprimorar suas abordagens para garantir uma manipulação de texto mais refinada e uma análise mais significativa. No entanto, a implementação ética e culturalmente sensível é essencial para preservar a riqueza e a diversidade da linguagem humana.

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu a como implementar o NLTK para tokenizar as sentenças do nosso dataset e observou o quão poderoso isso foi para nossa análise. Com a tokenização, observamos várias recorrências de stop words e aprendemos diversas formas de retirá-los do projeto deixando nossos gráficos muito mais interessantes para tomada de decisão.



REFERÊNCIAS

ANDRADE, A.C. **Relação da Linguística com o Processamento de Linguagem Natural**. 2021. Disponível em: <<https://www.alura.com.br/artigos/relacao-linguistica-processamento-de-linguagem-natural>> Acesso em: 17 jan. 2024.

NLTK. **Documentação NLTK**. [s.d]. Disponível em: <<https://www.nltk.org/>>. Acesso em: 17 jan. 2024.

SEABORN. **Documentação Seaborn**. [s.d]. Disponível em: <<https://seaborn.pydata.org/>> Acesso em: 17 jan. 2024.

PALAVRAS-CHAVE

Tokenização. NLTK. StopWords.

EMSE



POSTECH